

SpinalSynth: Envelope Generator Specification

Description

The **Envelope Generator** is a control module designed to shape the volume (amplitude) or other modulation parameters of a sound over time. The general design principle is an ADSR engine. In a nutshell, this is how it works:

When a key is pressed (Gate ON), the envelope rises to peak volume (Attack), decays slightly to a steady volume (Decay and Sustain), and then fades to silence when the key is released (Release).

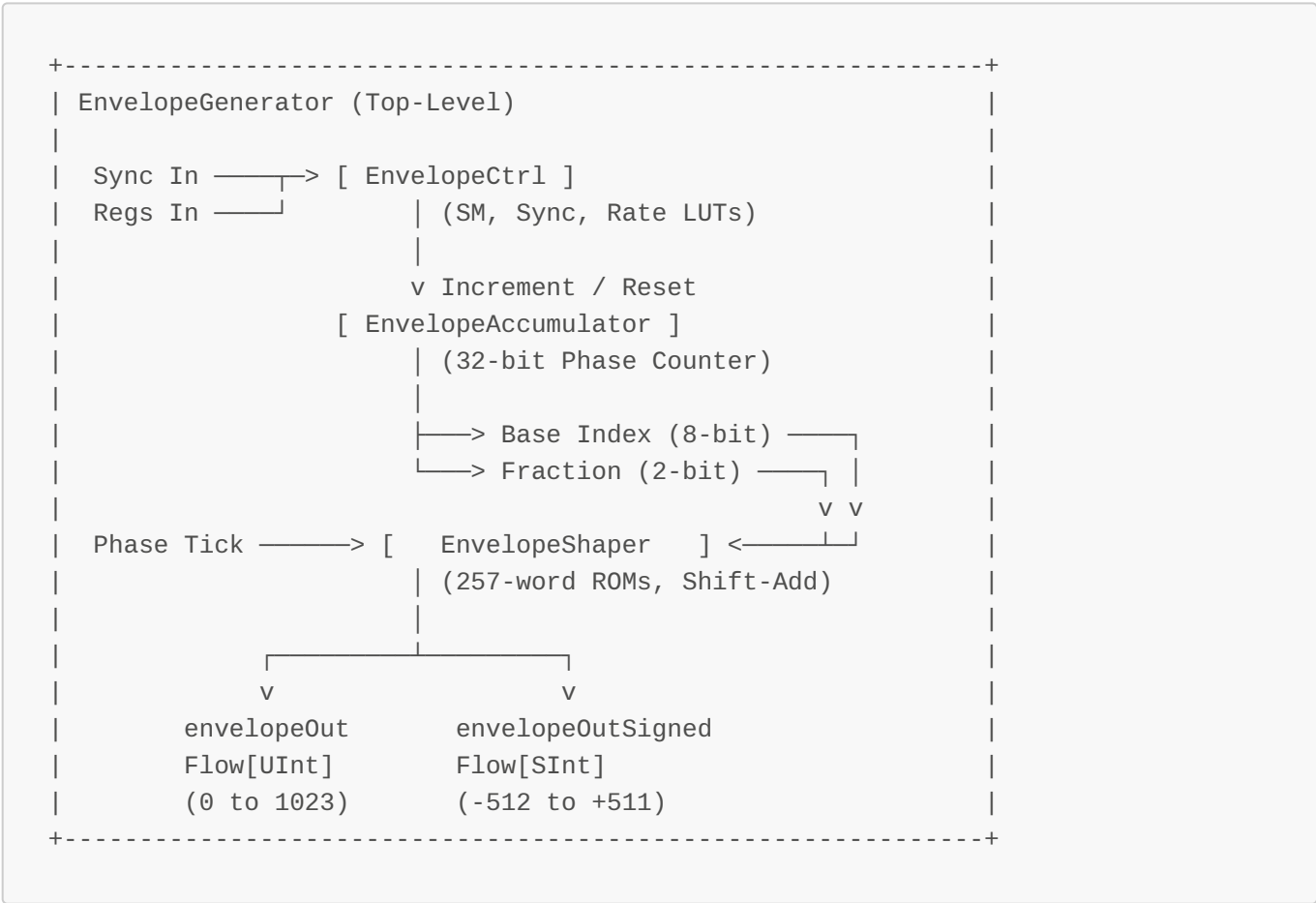
This module generates envelopes with a 10-bit resolution (0 to 1023) output value, which can be used as a volume (or other modulation) signal.

The entire module is designed with ASIC portability in mind, meaning it uses no specific hardware multipliers or memory blocks. Instead, it relies on compile-time Scala calculators to generate look-up curves in ROM, and performs most intermediate steps using bit-shifts and additions.

1. EnvelopeGenerator: Top-Level Wrapper

The top-level **EnvelopeGenerator** module integrates the submodules and registers them to the system communication and audio pipelines.

System Diagram



Module Interface (I/O Ports)

The top-level **EnvelopeGenerator** operates directly on the 24 MHz main system clock and exposes the following SpinalHDL hardware IO bundle:

```
val io = new Bundle {
  // Inputs
  val phaseTick = in Bool() // Heartbeat tick synced with 480 kHz
sample rate
  val syncIn    = in Bool() // External trigger for Hard or Soft
Sync
  val midiClock = in Bool() // External MIDI clock tick (24 PPQN
pulse)
  val config    = in(EnvelopeConfig()) // Packaged register configurations

  // Outputs
  val envelopeOut      = master(Flow(UInt(10 bits))) // Unipolar output (0 to
1023)
  val envelopeOutSigned = master(Flow(SInt(10 bits))) // Bipolar output (-512 to
+511)
}
```

- **Unipolar Output (envelopeOut):** Emits unsigned 10-bit values (0 to 1023) for standard amplitude scaling or unipolar modulation. The flow's `valid` signal is synchronized to `phaseTick` (480 kHz heartbeat).
- **Bipolar Output (envelopeOutSigned):** Emits signed 10-bit values (-512 to +511) for ring modulation, phase modulation, or center-zero pitch modulations. The flow's `valid` signal is synchronized to `phaseTick` (480 kHz heartbeat).

The EnvelopeConfig Bundle

Following the consistent design patterns of the synthesizer's components, the parameter configuration is packaged into a unified Scala bundle under the `synth.common` package:

```
case class EnvelopeConfig() extends Bundle {
  val ctrl      = Bits(8 bits)
  val attack    = UInt(8 bits)
  val decay     = UInt(8 bits)
  val sustain   = UInt(8 bits)
  val release   = UInt(8 bits)
  val syncCtrl  = Bits(8 bits)
  val phaseOffset = UInt(8 bits)
}
```

Register Map

The following registers are mapped into the `spinalSynth` SPI/UART register bus to control the Generator parameters:

Register Address (Hex)	Register Name	Bit Width	Description
0x40	ENV_CTRL	8 bits	Control bits: [0] Enable, [1] Gate, [2] Loop, [3] Ping-Pong, [4] Reverse, [6:5] Curve Model (00=Lin, 01=Exp, 10=Log, 11=S-Curve)

Register Address (Hex)	Register Name	Bit Width	Description
0x41	ENV_ATTACK	8 bits	Attack rate coefficient (speed of phase accumulator in Attack)
0x42	ENV_DECAY	8 bits	Decay rate coefficient
0x43	ENV_SUSTAIN	8 bits	Sustain Level (0 to 255, scaled to 10-bit range internally)
0x44	ENV_RELEASE	8 bits	Release rate coefficient
0x45	ENV_SYNC_CTRL	8 bits	Sync config: [0] Hard Sync Enable, [1] Soft Sync Enable, [2] MIDI Sync Enable, [6:3] Clock Division Rate
0x46	ENV_PHASE_OFFSET	8 bits	Phase offset value (0 to 255 representing 0 to 360 degrees)

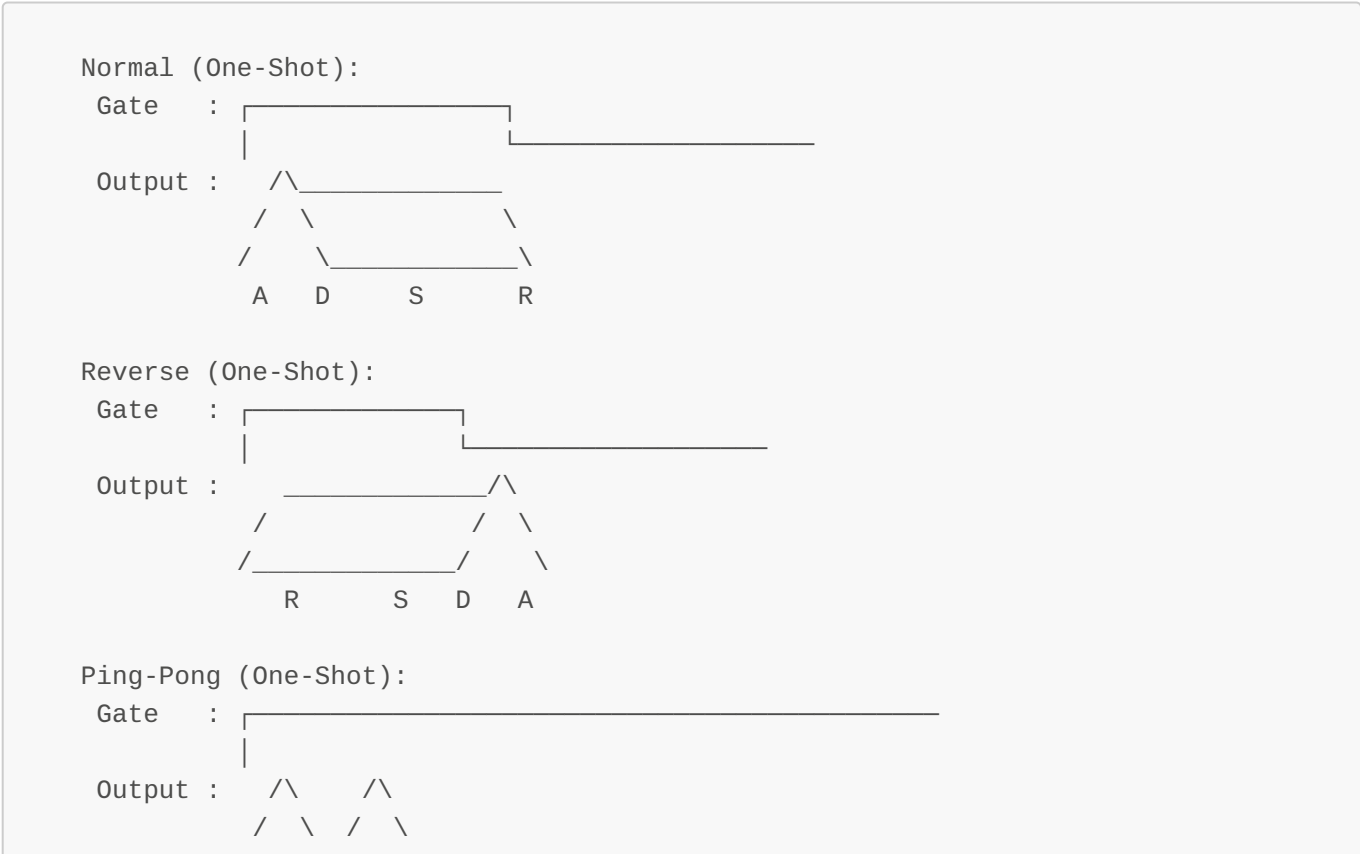
2. EnvelopeCtrl

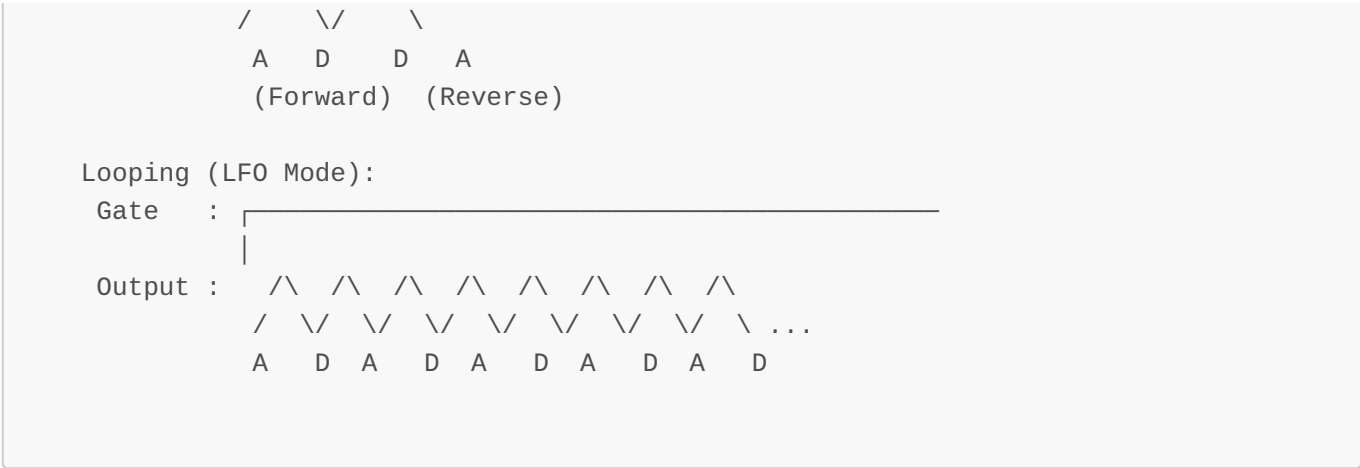
EnvelopeCtrl1 is the state machine and synchronization module that determines the active phase increment values and the play direction.

2.1 ADSR & Advanced Playback Modes

There is different modes for the ADSR playback envelopes and shapes:

- **Normal (One-Shot):** Triggers on Gate ON, transitions from Attack to Decay to Sustain, and goes to Release on Gate OFF.
- **Reverse (One-Shot):** Reverses the playback of active segments.
- **Ping-Pong Mode (One-Shot):** Playback alternates direction: playing forward (Attack -> Decay) and then reversing (Decay -> Attack in reverse shape). The complete envelop is therefore double the single length.
- **Looping (LFO Mode):** The envelope automatically loops back to the start of the Attack phase once the Decay phase finishes.





2.2 Sync and Phase

- **Hard Sync:** An external sync trigger instantly resets the current envelope phase accumulator back to 0 (start of Attack) and restarts the state machine.
- **Soft Sync:** An external sync trigger immediately switches direction to falling (if rising), or scales rate to phase-lock to the trigger frequency without clicky resets.
- **MIDI Sync & Clock Division:** Locks Attack, Decay, and Release times to tempo-subdivisions of MIDI Clock (24 Pulses Per Quarter Note - PPQN), with a 0 to 360-degree Phase Offset.

2.3 AD(S)R Lengths: Time Duration Mapping

In synthesizer design, how parameter values map to actual time durations directly determines the musical feel of the instrument.

Why Linear Mapping Fails

If we map the 8-bit parameters (0 to 255) of the Attack, Decay and Release registers to time durations linearly, we encounter severe playing issues:

- **Linear Time Mapping:** If time increases linearly up to 30.0 seconds, the first step is already 117 milliseconds. This completely wipes out the ability to dial in snappy, high-energy percussion attacks (which require precise control between 1 ms and 50 ms).
- **Linear Increment Mapping:** If we instead map the accumulator's step size (increment) linearly, we get a hyperbola where most of the knob range is crammed into tiny millisecond adjustments at the fast end, making it practically impossible to select slow durations with any precision.

The Logarithmic Solution

To match human hearing perception for both fast snaps and slow sweeps, we use a **logarithmic time mapping** (exponential increments). This splits the 8-bit parameter range into three playable musical zones:

- **Register Values 0 to 100:** Snappy transients (0.5 ms to 200 ms) with sub-millisecond precision.
- **Register Values 100 to 200:** Medium decay and release controls (200 ms to 3.0 seconds).
- **Register Values 200 to 255:** Very slow, evolving ambient sweeps (3.0 seconds to 30.0 seconds).

Hardware Implementation: Pre-Calculated ROM

Calculating logarithmic curves or exponential step values at runtime is expensive in ASIC silicon, requiring division blocks and exponential math units.

To maintain ASIC portability, we pre-calculate the 256 increment step values in Scala at compile-time. When a parameter register (Attack, Decay, or Release) is written, the system simply uses the 8-bit value to index a highly

efficient, static lookup ROM (256 words x 22-bit width) to retrieve the accumulator step size instantly.

```
System Specifications:
mainClock           = 24 MHz system clock
T_min               = 0.5 ms (0.0005 seconds)
T_max               = 30.0 seconds
Accumulator Width  = 32 bits (10 bits integer + 22 bits fraction)
Increment Width     = 22 bits

Mathematical Model:
T(P)                = T_min * (T_max / T_min) ^ (P / 255)
increment(P)        = 2^32 / (T(P) * 24,000,000)
```

3. EnvelopeAccumulator

The **EnvelopeAccumulator** acts as the time-tracking motor of the envelope generator.

How the Accumulator works

Basically the accumulator is a just register with a fixed width of 32 bits. On every system clock cycle (24MHz), the accumulator adds the selected 22-bit phase increment to its current value. The upper 10 bits of the accumulator are used as the output. These 10 output bits form a ramp function, like a sawtooth wave. The frequency of this ramp is determined by the phase increment value.

Clock and Frequency Boundaries

At the 24 MHz main clock rate with a 32-bit accumulator and 22-bit phase increment, the exact operational limits are calculated as follows:

Target Speed Limit	Time Duration	Active Clock Cycles	Calculated Increment (Decimal)	Increment (Hexadecimal)
Maximum Speed (T_min)	0.5 milliseconds	12,000 cycles	357,914	0x05761A
Minimum Speed (T_max)	30.0 seconds	720,000,000 cycles	6	0x000006

Hardware Phase Specifications

- **Accumulator Size:** Uses a 32-bit phase accumulator (10 bits integer + 22 bits fraction).
- **Segment Limits:** When the phase sweeps to its terminal value (overflow, depending on direction), the module asserts the **segmentComplete** flag back to **EnvelopeCtrl** to prompt state transitions.
- **Output Splitting:** Splits the upper 10 integer bits of the active 32-bit phase (bits 31 to 22) into two fields to drive the waveshaper:
 - **Base Index:** The higher 8 bits of the integer part (bits 31 to 24), representing the active step index (0 to 255).
 - **Fractional Part:** The lower 2 bits of the integer part (bits 23 to 22), representing the interpolation fraction (0 to 3).

4. EnvelopeShaper

The **EnvelopeShaper** is the output stage of the envelope generator.

It takes the raw, linear sawtooth ramp output from the accumulator (which always counts from 0 to 1023) and transforms it into musically expressive envelope curves.

It uses the upper 8-bit to lookup the active curve shape from our small ROM tables, and uses the 2 fractional bits to smoothly fill in the gaps between the data points. All interpolation is performed in pure combinational shift-add logic without any physical multipliers.

Finally, it outputs the unipolar (unsigned Int) and bipolar (signed Int) signals in parallel.

4.1 ROM Lookup tables

The higher 8 bits of the 10-bit accumulator output (Base Index) are used as the address to lookup the selected curve shape from our small ROM tables. The curves are pre-calculated as 257-word ROMs (257 x 8 bits) using these profiles:

Curve Model	Description	Primary Audio Application
Linear (Lin)	Perfectly straight transition lines.	LFO sweeps, pitch modulation, physical modeling.
Exponential (Exp)	Accelerating curve start, mimicking natural capacitor discharge.	Snappy percussion envelopes, natural string plucks.
Logarithmic (Log)	Rapid initial rise followed by gradual flattening.	High-energy attack dynamics, volume compensation.
S-Curve (Sigmoid)	Smooth cosine-like ease-in and ease-out transitions.	Smooth organic sweeps, cinematic pads, crossfading.

The 257-Entry ROM Boundary Safeguard

To calculate $Y1 = LUT[x+1]$ when the base index is at its boundary ($x = 255$) without conditional bounds checking or wrapping, the curve ROM is constructed with **257 entries** (indices 0 to 256). For $x = 255$, $LUT[x+1]$ safely returns $LUT[256]$, containing the true terminal amplitude value.

4.2 Hybrid 8+2 Bit Interpolation Math

Using the splits from the 10 bits accumulator output:

-  and $Y1 = LUT[x+1]$.
- The 2-bit fraction f represents step fractions {0, 1/4, 2/4, 3/4}.
- **Interpolated Output Y:** $Y = Y0 + (f / 4) * (Y1 - Y0)$

4.3 Multiplierless Shift-Add Implementation

The fractional calculation is implemented in pure combinational shift-add logic:

Fractional Bits (f)	Fraction Value	Hardware Shift-Add Expression
00	0.00	$Y0$
01	0.25	$Y0 + (\text{delta } Y \gg 2)$
10	0.50	$Y0 + (\text{delta } Y \gg 1)$
11	0.75	$Y0 + (\text{delta } Y \gg 1) + (\text{delta } Y \gg 2)$